

把需求往供应链下游传时哪些参数不能丢 Ontology-based Requirements Transformation

本篇范围声明：这是一篇 8 页的会议论文。下面的「全文翻译」按章节推进，覆盖摘要与正文第 I - VI 节的主体（引言、背景与相关工作、需求、方案、两个用例、结论），两张 UML 图和一张流程图保留其结构说明，Scala 代码模板保留骨架并逐段说明；文末致谢与 25 条参考文献列表不逐条翻译。

△这不是一篇 2026 年的新论文，详见「背景补充」第一节。

核心内容

Q：这篇要解决的是一个什么问题？ A：汽车零部件的可靠性需求，是靠一份叫 **Mission Profile（任务剖面，下称 MP）** 的东西往供应链下游传的。MP 记的是这个部件一生要承受什么——环境载荷（比如 150° C 下 100 小时）和功能载荷（电压随时间的波形）。

问题出在**每传一层就要变换一次，而变换会丢信息**。整车厂（OEM）手上是完整的，传给一级供应商（Tier-1）时按它的需要裁一份，**裁完之后 Tier-1 就没法再往 Tier-2 传了**——因为 Tier-2 要的那部分可能在第一次裁剪时已经被扔掉了。结果是每一层都**得从原始 MP 重新裁一次**，而不是顺着链条往下传。

Q：他们的解法是什么？ A：用 OWL 本体加 SWRL 规则，让推理机算出「这一次变换里，哪些参数不能丢」。

核心机制是一个失效模型本体。里面登记了每个失效模型用到哪些 MP 参数。**当某个参数是「下游某一层要用的失效模型」的输入时，推理机就把它标成必须保留**。人只需要指定「我现在在供应链的哪一层做这次变换」，剩下的推理机来答。

Q：为什么这件事非要用本体和推理，不能写个脚本？ A：论文给的理由是**数据源异构**。MP 现在没有统一格式，各家用 Excel 或 Word 传，**自动变换需要为每一种表示各写一套定制方案**。用 OWL 这个标准语言统一表示之后，才谈得上跨项目复用和跨异构系统的知识传递。

但更关键的一点论文没有明说，我认为它才是这个设计的核心价值：**「某个参数是否必须保留」这条知识，被登记在失效模型那一侧，而不是写在每一次变换里**。新增一个失效模型，所有相关的变换自动就知道要多保留一个参数——不需要去改任何一次变换的定义。

Q：他们怎么衡量这个系统做成了没有？ A：先提三条假设（要考虑供应链、载荷描述要通用、详细程度会变化），从假设推出五条抽象关键需求，再把五条需求落成五条可判定的验收标准（EC）：

标准	要求
EC.Parameter	系统含有维护「后续层级所需参数」的机制

标准

要求

EC.Transform 系统至少包含两种属于文献中不同类别的变换方法

EC.Extension 系统提供扩展新变换方法的机制

EC.Modify 系统提供的变换方法，其变换行为可修改

EC.Detail 系统对不同详细程度的 MP 数据处理得同样好

这套「从假设到需求到可判定标准」的推进方式本身值得抄，论文原话是「这种逐级细化反映了我们的工作过程」。

Q：跑出来的效果有数字吗？ A：有数字，但那些数字量的是规模不是效果。用例一给了变换前后的本体度量：公理 3874 → 3633、类 52 → 36、对象属性 39 → 31、数据属性 30 → 19、个体 775 → 774。而对象属性断言 779 → 779、数据属性断言 1216 → 1216，一条没少。

作者用这两个「不变」来说明变换保住了实际数据，丢掉的公理是有意丢弃的 MP 信息（工况状态、端口规格）。这是个诚实的展示，但它证明的是「没丢该留的」，不是「这套系统带来了收益」。引言里列的五项收益（保护知识产权、减少沟通开销、跨项目复用、避免错误、降低成本）一项都没有量化。

Q：作者自己承认的限制是什么？ A：两条，第一条是真限制。SWRL 只支持单调推理——基于 SWRL 规则的变换只能往本体里加事实，既不能修改也不能删除已有信息。而 MP 变换的本质需求之一恰恰是删减（裁掉下游不需要的参数以保护知识产权）。他们的缓解办法是给 SWRL 变换加前置和后置处理步骤，在那里做修改和删除。

第二条是实现层面的：本体模型是在内存里构建的，对特别大的 MP 会有问题。作者补了一句「这在实践中不常出现」。

背景补充 / 点评

先说一件必须说的事：这是 2019 年的论文

arXiv 编号 2608.21945，2026 年 8 月 22 日提交。但正文第 105 行的页脚写得很清楚：

This is the accepted manuscript of: J. Novacek, A. Viehl, O. Bringmann, and W. Rosenstiel, "Ontology-based Requirements Transformation," in **2019 IEEE International Symposium on Systems Engineering (ISSE)**, pp. 1 - 8, 2019. DOI: 10.1109/ISSE46696.2019.8984265

所以它是七年前的工作，2026 年只是把录用稿传到了 arXiv 上。

而且这是同一批里的第二次。2026-08-28 那批里的《Ontology-supported Design Parameter Management for Change Impact Analysis》，arXiv 编号 2608.21949，查下来是 Euromicro SEEA 2018 的录用稿。两个编号只差 4，说明是同一次批量上传，而且很可能来自同一个研究组（FZI Karlsruhe 与 Tübingen 大学）在集中补传旧的录用稿。

这件事对流水线有直接影响：抓取脚本按 arXiv 提交日期筛新文章，筛不出「新提交的旧论文」。这一批里可能还有更多。读的时候要留意页脚的 DOI 和会议年份。

旧不等于没价值——2019 年的 OWL、SWRL、Protégé 到今天仍然是标准工具，这篇讲的机制不过时。但**「近期研究进展」这个判断不能建立在它上面**，而且它引的相关工作停在 2018 年。

几个术语，先说清楚

Mission Profile (MP, 任务剖面)：记录一个部件在整个生命周期中所承受的环境条件与功能载荷的文档。汽车行业用它把可靠性需求沿开发链条往下传。它的数据来源是载荷使用剖面、环境剖面、需求，以及应力测试场景。**目前没有统一标准**——RESCAR2.0 项目做过初稿，autoSWIFT 项目在推一个叫 **MPFO (Mission Profile Format) **的格式。

MPAD (Mission Profile Aware Design, 任务剖面感知设计)：Jerke 和 Kahng 提出的概念。论文写这一节时的现状是：**MP 的考量基本还是人工活**，作者说他们只知道一个支持 MPAD 的框架——恩智浦的 Reliability Knowledge Framework。支持 MPAD 的软件之所以稀少，可能是因为 MP 标准还不存在。

MBSE (Model-based Systems Engineering, 基于模型的系统工程)：用模型而不是文档来做系统设计的方法论，在汽车和航空航天行业已被接受为设计复杂系统的有效方式。这篇的定位是**能接进已有的 MBSE 流程**，不是另起一套。

M2M (Model-to-Model, 模型到模型) 变换：文献把它分成关系式/声明式、命令式/操作式、基于图的、混合式四类。**这篇故意支持多类而不限定一类**，因为「可扩展」被列为关键需求之一。

SWRL (Semantic Web Rule Language)：语义网规则语言，在 OWL 之上加规则表达能力。**它只支持单调推理**——这是它的定义性质，不是实现缺陷：规则只能推出更多为真的事实，不能推翻已有事实。

FEOL / BEOL：半导体制造的前道工序 (Front-End-Of-Line, 做晶体管) 和后道工序 (Back-End-Of-Line, 做金属互连)。论文的失效模型按这两道分：FEOL 的 TDD (时间相关介质击穿)、HCI (热载流子注入)、BTI (偏置温度不稳定性)，BEOL 的 TDD 和 EM (电迁移)。

JEDEC JESD51-2：一个测量集成电路热阻的标准方法。用例二用到它的公式： $\theta_{JA} = (T_J - T_A) / P$ ，反过来写就是 $T_J = T_A + P \times \theta_{JA}$ ——已知环境温度、器件总功耗、结到环境的总热阻，就能算出结温。

那两条 SWRL 规则值得单独看

论文给了两条规则，**它们的差别是「直接用到」和「间接用到」**：

规则一 (MaintainParametersOEM)：如果存在一个技术级失效模型用到了某个 MP 参数，那么在 OEM 层做的这次变换就必须保留它。

```
MPOTransformation(trans) ∧ ofSupplyChainLevel(trans, OEMLevel) ∧  
TechnologicalFailureModel(fm) ∧ hasMissionProfileParameter(fm, param)  
→ maintainsParameter(trans, param)
```

规则二 (MaintainParametersHighLevel)：高层失效模型也可能间接需要技术层的参数。判据是这个失效模型所对应的失效，它的根失效是否落在技术层：

```
... HighLevelFailureModel(fm)  $\wedge$  ofFailure(fm, failure)  $\wedge$ 
hasRootFailure(failure, rootFailure)  $\wedge$  TechnologicalFailure(rootFailure)  $\wedge$ 
hasMissionProfileParameter(fm, param)
 $\rightarrow$  maintainsParameter(trans, param)
```

第二条才是这套机制真正的价值所在。第一条人也能查——列个表就行。第二条要沿着「失效 $\rightarrow$ 根失效 $\rightarrow$ 是不是技术层失效」这条链走，而这条链有多长事先不知道。**这正是「用推理而不用查表」的理由**：依赖关系是传递的，深度不定。

论文还明确写了一句实施注意事项：**这两条规则要为供应链的每一层各定义一份**

(ofSupplyChainLevel(trans, OEMLevel) 那个常量得换)。这意味着供应链层数变了就要改规则——这个设计本身有个可维护性缺口，论文没有讨论。

我的判断

这篇最值得学的不是它做了什么，是它把知识登记在哪一侧。

「某个 MP 参数必须保留」这条知识，有两个地方可以放：写在每一次变换的定义里（用途那一侧），或者登记在失效模型上（事实那一侧）。**这篇选了后者。**

后果是可验证的：新增一个失效模型，只要登记好它用到哪些参数，**所有层级的所有变换自动就知道要多保留一个参数**，一次变换定义都不用改。反过来如果写在变换里，加一个失效模型就要去翻所有变换定义，漏一个就静默地丢参数——**而且不报错**，只是下游某一层的可靠性计算少了一个输入。

这个形状我在 prove 项目里刚好碰到过反面案例：指标注册表里 completion_not_deleted 这个过滤器被 10 个指标各 use 了一遍，exam_submitted_only 9 次。而「完成度这张表永远要排除已删除的行」是关于那个**实体**的事实，不是关于那 10 个指标各自的事实。写在用途侧的后果是：新加第 11 个用到 completion 的指标，忘了写 use，数字就多算已删除的行，不报错。

这篇给了一个把它做对的成功案例，而且给了做对之后能拿到什么：不是省了几行配置，是**新增知识时不用回头改所有消费者**。

三处我认为站不住的地方

第一，五项收益一项都没测。引言里列的「保护知识产权、减少沟通开销、跨项目复用变换、避免变换过程中的错误、降低成本」，全篇没有任何一项给出度量。用例里唯一的数字是本体度量表（公理数、类数、个体数），**那些数字量的是本体的规模，不是这套方法的收益。**

要测其实不难：「减少沟通开销」可以数「需要几次变换」——论文自己在图 3 里画了改进前需要 (A)(B)(C) 三条路径，改进后应该少几条，这个数完全可以给出来但没给。

第二，五条验收标准里有四条是「存在性检查」。EC.Parameter「含有一个机制」、EC.Transform「至少包含两种」、EC.Extension「提供一个机制」、EC.Modify「行为可修改」——这四条问的都是「有没有」，不是「好不好用」。一个能跑通最小例子的实现就能全部通过。

第五条 EC.Detail 更成问题：它要求「对不同详细程度的 MP 数据处理得同样好」，而论文对应的回答是「系统只依赖已知性能良好的成熟库」，然后补一句「具体的变换定义对性能有直接影响，所以这主要取决于用户」。这等于没有回答——把标准的达成责任推给了使用者，那这条标准就不可判定。

第三，SWRL 的单调性限制是被绕过去的，不是被解决的。作者承认 SWRL 只能加不能删，缓解办法是「加前置和后置处理步骤，在那里做修改和删除」。这意味着变换逻辑被劈成了三段：前处理（命令式代码）、SWRL 规则、后处理（命令式代码），而只有中间那段是声明式、可推理、可复用的。那么「用本体表达变换」这个卖点，覆盖的到底是多大一部分？论文没有说。

而 MP 变换的核心动作恰恰是删减——「裁掉下游不需要的参数以保护知识产权」是它自己列的第一项收益。所以最重要的那个动作，落在了 SWRL 管不到的地方。

对我的启示

第一条，可以直接搬到 prove 上。

我们的语义层里，指标编译成 SQL 的过程也是一次「变换」，而且也有「哪些东西必须保留」的问题——比如标识符必须参与数字对账、学号不许从 gate 豁免（这条是我们踩过坑记下来的）。现在这条知识写在哪里？写在人的记忆和文档里。

这篇的做法可以照抄：把「这个字段在下游还有消费者」登记成结构化事实，让程序去查，而不是靠人记。具体到我们：mapping-registry 里每个实体的 fields 已经逐列登记了，缺的是「这一列被哪些下游用到」这个反向索引。有了它就能回答「这次改动会不会让某个下游算错」——而不是等对账数字不一致了再回头查。

第二条，「传递依赖必须用推理，不能用查表」这个判据。

那两条 SWRL 规则的差别给了一个清楚的判据：依赖关系如果是一跳，查表就够；如果是传递的、深度不定的，就必须让程序去走那条链。

规则二走的是「失效 → 根失效 → 是不是技术层」，链长不定。我们那边对应的是 scope / attach / compose / from_query 的嵌套——如果哪天要回答「改这个实体会影响哪些指标」，那也是一条深度不定的链，不能靠维护一张手工映射表。

第三条是关于怎么读验收标准的，这条更通用。

这篇的五条 EC 里四条是存在性检查、一条把责任推给用户。「系统含有一个机制」这种表述读起来很像验收标准，但它的通过条件是「有没有写这段代码」，不是「这段代码有没有用」。

以后我自己写验收标准，或者审别人的，要问一句：这条标准，一个只能跑通最小例子的实现能不能通过？能通过，它就不是验收标准，是功能清单。

文中金句

"A limitation of the SWRL rule-based transformation is that SWRL supports monotonic inference only. Because of that SWRL-based transformations can only add more facts to an ontology but neither modify nor remove existing information." 基于 SWRL 规则的变换有一个限制：SWRL 只支持单调推理。因此，基于 SWRL 的变换只能往本体里添加更多事实，既不能修改也不能移除已有信息。

"Subsequent transfer of MP data from Tier-1 to Tier-2 is not possible as there might be information missing due to the initial transformation. Therefore, a cumbersome process involving transformations of the base MP to each tier's demands is necessary." 从 Tier-1 向 Tier-2 继续传递 MP 数据是不可能的，因为最初那次变换可能已经把某些信息丢掉了。因此必须走一个繁琐的流程：把基准 MP 分别变换成每一层各自需要的样子。

"The idea is to mark parameters as required when they are parameters of a failure model that can be applied on a lower tier." 这个想法是：当某个参数是「某个可以在更低层级上应用的失效模型」的参数时，就把它标记为必须保留。

"Of course concrete definitions of transformations have a direct impact on performance. Therefore, it is mostly up to the user to keep that in mind while defining transformations." 当然，具体的变换定义对性能有直接影响。因此这主要取决于用户在定义变换时是否留意这一点。（——这句是第五条验收标准的全部回答，等于把标准的达成责任交给了使用者。）

全文翻译

摘要

This paper presents an ontology-based approach to the supply chain-aware transformation of functional and environmental load requirements given by so-called Mission Profiles (MPs). The approach aims at improving the efficiency of the engineering process through supporting the transformation process and enabling a better integration of the transformation into existing Model-based Systems Engineering (MBSE) processes.

本文提出一种基于本体的方法，用于对所谓 **Mission Profile (MP, 任务剖面)** 给出的**功能载荷与环境载荷需求做供应链感知的变换**。该方法的目标是：通过支撑变换过程、并让变换能更好地融入既有的****基于模型的系统工程 (MBSE) ****流程，来提升工程过程的效率。

We propose a methodology and a supporting system which aids in the transformation process while the latter feature is obtained by constructing and working on models. Consequent utilization of the standardized language OWL to express model representations further enables better knowledge integration and transfer among heterogeneous systems.

我们提出一套方法论和一个配套系统来辅助变换过程；而「融入 MBSE」这一点是通过构建模型、并在模型上工作而获得的。持续使用标准化语言 **OWL** 来表达模型表示，进一步使得异构系统之间的知识集成与传递更为顺畅。

In addition to that, this favors knowledge reuse across projects which can reduce overall costs. Moreover, the system enables stripping off irrelevant information from MPs, thus improving protection of intellectual property.

除此之外，这也有利于知识在项目之间复用，从而降低总体成本。此外，该系统能够**从 MP 中剥离无关信息，从而加强对知识产权的保护。**

索引词：系统工程、基于知识的工程、需求管理、任务剖面、模型到模型变换。

一、引言：需求沿供应链下传时会丢信息

One important aspect of the tremendous increase of system complexity is the resulting necessity to manage huge amounts of requirements efficiently. This includes the processing of requirements for transforming them as they are broken down to separate tier needs.

系统复杂度急剧上升带来的一个重要后果，是必须高效地管理数量庞大的需求。这包括在需求被分解成各个层级各自的需要时，对它们进行变换处理。

Ontologies are used to capture knowledge and to represent models to enable a better integration of heterogeneous data sources and to encourage reuse of knowledge across project life cycle phases and between projects which has been identified as an important aspect of today's Systems Engineering challenges.

我们用本体来捕捉知识、表示模型，以便更好地集成异构数据源，并鼓励知识在项目生命周期各阶段之间、以及在项目与项目之间复用——这已被指认为当今系统工程挑战中的一个重要方面。

Typically, requirements are passed along the supply chain while being refined and broken down to subsequent tier needs. In this process MP data needs to be transformed. The selection of an appropriate transformation, its parametrization, and applying it to the MP data can be supported by using ontologies and reasoning.

通常，需求沿供应链传递，同时被细化并分解成后续层级的需要。在这个过程中 MP 数据需要被变换。**选择合适的变换、给它设定参数、以及把它应用到 MP 数据上，这三件事都可以用本体和推理来支撑。**

The design of complex systems imposes several engineering challenges. A predominant problem is that Systems Engineering data is primarily stored in documents that are scattered across engineering workstations, and poorly managed. With regard to requirements, MPs are shared in a non-standardized way for example by using Excel sheets or Word documents. Transforming them automatically is therefore challenging as it requires implementing a custom solution for each MP representation.

复杂系统的设计带来若干工程挑战。一个突出的问题是：系统工程数据主要存放在文档里，这些文档散落在各个工程工作站上，管理很差。就需求而言，**MP 是以非标准化的方式共享的，比如用 Excel 表或 Word 文档**。因此自动变换它们很困难，因为那需要为每一种 MP 表示各实现一套定制方案。

主要收益（论文原文列举五项）：

- 由于移除了不必要的信息，**对知识产权（IP）的保护更好**
- 由于变换次数减少，**沟通开销降低**
- **变换可以跨项目更好地共享和复用**
- **避免变换过程中的错误**
- 由于复杂度下降，**成本降低**

As all models used in the presented approach are ontology-based, we follow the principle of Linked Data that allows interlinking and discovery of new data sources in a global data graph. Relying on an established standard for an ontology language, the system also favors sharing and processing of system models used in development.

由于本方法中用到的所有模型都是基于本体的，我们遵循 **Linked Data** 的原则——它允许在一个全局数据图中互联并发现新的数据源。依托一个成熟的本体语言标准，该系统也有利于开发中所用系统模型的共享与处理。

（**图 1** 说明的是变换的必要性：一个只有单一安装点的部件，其 B 端口的功能载荷与环境载荷与整体不同——同一个部件，安装点处的环境载荷是「150° C 下 100 小时」，而 B 端口处是「175° C 下 100 小时」，所以必须做变换。）

二、背景与相关工作

A. Mission Profile

Capturing and specification of environmental conditions and functional loads which components are exposed to throughout their life cycle is done via MPs. Therefore, they play an important role in the design and development of automotive systems, especially with regard to the propagation of requirements along the development process chain.

对部件在整个生命周期中所承受的环境条件与功能载荷的捕捉和规定，是通过 MP 完成的。因此 MP 在汽车系统的设计与开发中扮演重要角色，尤其是在需求沿开发过程链传播这件事上。

To create a MP various sources are used such as load usage and environmental profiles, requirements and scenarios for stress tests. An initial draft of a format for MPs was developed in the RESCAR2.0 project. To improve the collaboration among partners involved in automotive component design the autoSWIFT project aimed to prepare a standard for MPs, the Mission Profile Format (MPFO).

创建一份 MP 要用到多种来源：载荷使用剖面与环境剖面、需求、以及应力测试的场景。MP 格式的初版草案由 RESCAR2.0 项目制定。为改善汽车部件设计相关各方之间的协作，autoSWIFT 项目致力于准备一个 MP 标准，即 MPFO (Mission Profile Format)。

B. 任务剖面感知设计 (MPAD)

Jerke and Kahng introduced the general concept of MPAD. They emphasized that MP consideration is still mainly a manual task. This is why it is important to provide tool support to enable automation and ease adoption. Currently, we are only aware of a single framework supporting MPAD, the Reliability Knowledge Framework by NXP. The rareness of software supporting MPAD might be due to the fact that a MP standard does not exist yet.

Jerke 与 Kahng 提出了 MPAD 的一般概念。他们强调：MP 的考量目前仍然主要是一项人工任务。正因如此，提供工具支持以实现自动化、降低采纳门槛才显得重要。目前我们只知道一个支持 MPAD 的框架——恩智浦的 Reliability Knowledge Framework，它被创建出来是为了把用户、可靠性知识、数据、工具和方法连接起来。支持 MPAD 的软件之所以稀少，可能是因为 MP 标准尚不存在。

C. 模型到模型变换

While M2M transformation approaches can be divided into relational/declarative, imperative/operational, graph-based and hybrid categories, the approach to MP transformation presented in this paper supports multiple M2M transformation approaches and is not limited to one category. This is due to an identified key requirement.

M2M 变换方法可以分为关系式/声明式、命令式/操作式、基于图的、以及混合式四类。而本文提出的 MP 变换方法支持多种 M2M 变换方法，不限于其中某一类——这是因为我们识别出的一条关键需求要求如此。

三、需求：从三条假设到五条验收标准

In this section we list hypotheses from which we draw vital conclusions in form of identified abstract key requirements. Based on the key requirements, we also define evaluation criteria. This gradual refinement reflects our procedure.

本节列出若干假设，从这些假设中我们以「识别出的抽象关键需求」的形式得出关键结论。基于这些关键需求，我们再定义验收标准。这种逐级细化反映了我们的工作过程。

三条假设：

1) **供应链考量**：在不同层级上，需要的是原始 MP 数据的不同部分。比如要在技术供应商层级上应用失效模型，就要求在它之前的各层级维护住特定的 MP 参数，这种情况在更高层级上也会发生。**因此 MP 变换工具应当感知供应链。**

(图 3 画的就是这个问题：整车 → ECU → 电路 → 技术，对应 OEM → Tier-1 → Tier-2 → Tier-3。在 OEM 层级所有信息都以 MP 数据的形式可用。把基准 MP 变换给 Tier-1 通常得到的是 MP 信息的一个子集。**随后从 Tier-1 向 Tier-2 继续传递 MP 数据是不可能的**，因为最初那次变换可能已经丢掉了某些信息 (A)。因此必须走一个繁琐的流程：把基准 MP 分别变换成每一层各自的需要 (B)、(C)。

2) **通用的载荷规格**：为了给环境载荷的规定提供一个通用解法，描述载荷的手段必须足够多样，处理这些载荷规格的工具的功能集也一样。**发现此前未知的、会影响该部件的载荷，要求载荷规格机制及相关处理功能具备可扩展性。**

3) **详细程度会变化**：MP 的详细程度可能不同，因此处理 MP 数据的工具需要可伸缩。此外，处理 MP 数据的工具应当在**可定制的操作**方面具备适应性，以便同样好地处理不同的详细程度。

从假设中提取出的五条抽象关键需求：① 参数的维护机制；② 多样的处理功能集；③ 可扩展的处理功能集；④ 可定制的操作；⑤ 可伸缩性。

B. 验收标准

The level of abstraction of the former requirements makes it hard to decide what measures are suitable to meet a requirement because it is hard to tell whether it has been fulfilled or not. This also hinders a proper evaluation of the solution later on. To tackle these issues, we defined specific evaluation criteria EC associated with the key requirements which are more precise and specific.

前述需求的抽象层级太高，很难判断什么措施适合满足某条需求，因为很难说清它到底有没有被满足。这也妨碍了后续对解决方案做恰当的评估。为解决这些问题，我们定义了与关键需求相关联的、更精确具体的**验收标准 EC**：

标准	内容
EC.Parameter	变换系统包含一个机制，用于维护后续层级所需的参数
EC.Transform	变换系统至少包含两种变换方法，且它们按文献分类属于不同类别
EC.Extension	变换系统提供一个机制，可用新的变换方法来扩展它
EC.Modify	系统所提供的变换方法，其变换行为是可修改的
EC.Detail	变换系统应当同样好地处理不同详细程度的 MP 数据

四、方案

A. 概览

(图 2 的五步流程：❶ 人工指定所处的供应链层级；❷ 这个信息与已知失效模型一起，用于识别在变换中需要维护的参数；❸ 人工规定变换操作，产出可执行的变换操作；❹ 用第 IV-C 节描述的框架，把 MPFO 格式的基准 MP 文档映射成 MP 本体（MPO）；❺ 最后执行对基准 MPO 的、基于本体的变换。)

B. 方案概念

A fundamental characteristic of our proposed solution is continuous use of a unifying language to express models namely the ontology language OWL. The primary purpose of this is to alleviate difficulties and effort to integrate data, information and knowledge required to fulfill the task of MP transformation and support its implementation.

我们所提方案的一个根本特征是：**持续使用一种统一的语言来表达模型，即本体语言 OWL**。这样做的首要目的是减轻集成数据、信息和知识的困难与工作量——而这些数据、信息和知识正是完成 MP 变换任务、支撑其实现所需要的。

In order to obtain models expressed in OWL we suggest lifting existing models, by which we mean a) translation of the source model into suitable OWL expressions and b) enrichment of the resulting ontology with additional relevant knowledge to support task achievement.

为了得到用 OWL 表达的模型，我们建议对已有模型做**提升 (lifting)**，我们指的是两件事：a) 把源模型翻译成合适的 OWL 表达式；b) 用额外的相关知识丰富所得到的本体，以支撑任务完成。

1) 供应链感知：我们通过一个 MP 参数维护机制来实现对供应链的考量。**这个想法是：当某个参数是「某个可以在更低层级上应用的失效模型」的参数时，就把它标记为必须保留。**这一过程会参照系统中一个基于本体的失效模型库。

2) 多种变换方法：为区分变换方法，我们采用 Czarnecki 与 Helsén 的分类。为满足 EC.Transform，系统同时包含一种**直接操作式 (direct-manipulation) 和一种关系式 (relational) **的 M2M 变换方法。

3) 可扩展性：通过提供**插件机制**实现。该机制的具体设计取决于所选技术——我们用的是一门多范式（面向对象加函数式）编程语言，通过实现预定义的接口就能往系统里添加更多变换方法。

4) 可修改性：主要通过这样一个事实实现——按所选方法对一次变换所做的规定，本身就已经隐含了对行为的定义。此外变换方法可以被参数化，这有一个正面的副作用：鼓励复用。

5) 性能不随详细程度变化：为处理 MP 中不同的详细程度，系统只依赖已知性能良好的成熟库。**当然具体的变换定义对性能有直接影响。因此这主要取决于用户在定义变换时是否留意这一点。**

C. 框架设计

We utilized existing tooling of the Semantic Mission Profile Aware Design Platform to map MP documents to corresponding ontologies. The platform contains a tool which takes a MP document in MPFO document format as input and generates a corresponding OWL ontology which is serialized in RDF/XML format.

我们利用了 [Semantic MPAD Platform](#) 的既有工具链，把 MP 文档映射成对应的本体。该平台包含一个工具，它以 MPFO 格式的 MP 文档为输入，生成对应的 OWL 本体，并序列化为 RDF/XML 格式。

架构与实现：平台实现为一个 Play Framework 应用，用 **Scala** 语言编写，基于 RESTful 架构设计模式。设计意图有两条：一方面提供构建支持 MPAD 的应用的手段，另一方面通过提供一条务实的实现路径来加速开发。

核心功能：Semantic MPAD Platform 的首要用途是**处理、互联和集成 MP 与需求**。所有预期为文档的输入都由映射引擎提升为本体。平台还包含用于检测这些文档的变更、并对变更定义响应动作的工具。**用标准化本体语言 OWL 作为输出**，打开了把推理机制轻松集成进 MP 与需求处理流程的可能性。

接口：除 RESTful API 外，平台还提供基于 HTML 和 JavaScript 的网页图形界面，因此可以从任何符合标准的浏览器与系统交互，便于把系统集成进已有的设计流程与环境。

D. 架构

1) 失效模型与变换本体

To consider the supply chain and reach evaluation criterion EC.Parameter an ontology addressing the parameter maintenance is introduced. This ontology has the purpose of identifying parameters which need to be maintained through a transformation. The OWL ontology contains SWRL rules for the inference of parameters to maintain.

为考量供应链、达成验收标准 EC.Parameter，我们引入一个处理参数维护的本体。**这个本体的用途是识别出「在一次变换中需要被维护住」的参数**。该 OWL 本体包含用于推断待维护参数的 SWRL 规则。

****FailureModel（失效模型）****由公理 1 所表达的类的一个子类来建模：

```
∃hasEquation.xsd:string ⊑  
∃hasMissionProfileParameter.MissionProfileParameter ⊑  
∃ofFailure.Failure
```

****TechnologicalFailureModel（技术失效模型）****是失效模型的特化，因而是它的子类；同时也是公理 2 所表达的类的子类：

```
∃hasProductParameter.ProductParameter ⊑  
∃hasTechnologyParameter.TechnologyParameter ⊑  
∃ofFailure.TechnologicalFailure ⊑  
=1 hasLayer.Layer
```

****MPOTransformation（变换）****建模为一个类，它也是公理 3 所表达的类的子类：

```
≥ maintainsParameter.MissionProfileParameter ⊑  
=1 ofSupplyChainLevel.SupplyChainLevel
```

MaintainParametersOEM (规则 4): 当存在一个技术失效模型用到了这些参数时, 把

`MPOTransformation` 的 `OEMLevel` 实例上的 MP 参数设为必须维护。依供应链的不同, 必须为供应链的每一层类似地定义规则。

MaintainParametersHighLevel (规则 5): 我们还需要把「关于高层失效模型而言需要维护的所有参数」标记出来。规则 5 表明这可以通过检查某个失效模型所对应的失效是否有一个位于技术层的根失效来实现。

Furthermore, a taxonomy for failure models was extracted from the categories described by Maricau and Gielen. Complementary metal-oxide-semiconductor (CMOS) device failure models for Electromigration (EM), Hot-carrier injection (HCI), Time-dependent dielectric breakdown (TDDB) and Bias temperature instability (BTI) were added as instances of corresponding classes.

此外, 我们从 Maricau 与 Gielen 描述的类别中提取出一套失效模型的分类体系。*电迁移 (EM)、热载流子注入 (HCI)、时间相关介质击穿 (TDDB)、偏置温度不稳定性 (BTI) *这些 CMOS 器件失效模型, 被作为对应类的实例加了进去。

(图 4 是这个本体的分类体系。顶层 `owl:Thing` 下面有 `Technology`、`Layer`、`Failure`、`Parameter`、`FailureModel`、`MPOTransformation`、`SupplyChainTier` 等。其中 `Parameter` 下分 `MissionProfileParameter` (再分 `OperatingTemperaturesParameter`、`GeneralMissionProfileParameter`、`OperatingVoltagesParameter`)、`ProductParameter`、`TechnologyParameter`、`TransformationParameter`; `FailureModel` 下分 `HighLevelFailureModel` 和 `TechnologicalFailureModel`; `MPOTransformation` 下分 `DirectManipulationMPOTransformation` 和 `RuleBasedMPOTransformation`, 再往下是 `OWLAPITransformation` 和 `SWRLAPITransformation`; `Failure` 下分 `HighLevelFailure` 和 `TechnologyFailure`, 后者再分 `TransientFailure`、`AgingFailure`、`TemporalFailure`、`SpatialFailure`、`RandomFailure`、`SystematicFailure`。)

2) 系统

(图 5 是类型层次的 UML 图。要点: 用的是 Scala 的 **trait** 而非面向对象语言的接口——trait 允许多继承且可以有属性, UML 没有对应这种继承的语义概念。系统中每个对象都是一个 `TransformationObject`, 提供一个基于键值访问的、任意类型的 `metadata` 属性。)

To meet evaluation criteria EC.Transform and EC.Extension, we introduced a trait for transformations: `MPOTransformation`. This trait holds Mappings which themselves have sources and targets lists attributes. Designing this way allows the definition of one-to-one, one-to-many, many-to-one and many-to-many mappings.

为满足 EC.Transform 和 EC.Extension, 我们引入了一个变换的 trait: `MPOTransformation`。该 trait 持有若干 `Mapping`, 而 `Mapping` 自身有 `sources` 和 `targets` 两个列表属性。这样设计允许定义一对一、一对多、多对一和多对多的映射——这是 MP 变换中创造必要灵活性所需要的, 用以覆盖诸如「把若干 MP 合并成一个」这类用例。

`DirectManipulationMPOTransformation` 对象有一个 `modelProvider` 属性, 用于访问提供模型操作 API 的对象 (直接操作式变换方法需要它)。`RuleBasedMPOTransformation` 对象含有一个 `rules` 属

性，本质上是 Mapping 对象到 Rule 对象列表的映射——这允许把规则与特定映射精确关联起来。

为满足 EC.Modify，系统让用户通过在实现类或特化 trait 中重写变换 trait 的 apply 方法来定义变换行为。

五、两个工业用例

A. 技术失效模型的应用

To apply a failure model at the technology level, certain MP parameters need to be maintained in a transformation. We used a MP document in MPFO version 0.2.1 as an example which contained seven parameters (t0, t1, t2, v0, v1, v2, failureRate) and temperature ranges for stress testing a component expressed as eight intervals ranging from -10 ° C to 90 ° C.

要在技术层级上应用一个失效模型，某些 MP 参数必须在变换中被维护住。我们用一份 MPFO 0.2.1 版的 MP 文档做例子，它含有七个参数 (t0、t1、t2、v0、v1、v2、failureRate)，以及用于对部件做应力测试的温度范围——表示为从 -10° C 到 90° C 的八个区间（每个区间宽 10° C：-10~20、20~30、…、80~90）。

In our example, the ontology contains five front-end-of-line (FEOL) and back-end-of-line (BEOL) failure models FEOL-TDDB, FEOL-HCI, FEOL-BTI, BEOL-TDDB and BEOL-EM with corresponding technology, product and MP parameters defined. Utilizing reasoning, we found that temperature, voltage, and failure rate parameters needed to be maintained.

在我们的例子里，本体含有五个 FEOL 与 BEOL 失效模型——FEOL-TDDB、FEOL-HCI、FEOL-BTI、BEOL-TDDB 和 BEOL-EM，各自定义了对应的技术参数、产品参数和 MP 参数。这组 MP 参数包含电压参数、温度参数以及失效率。利用前述推理，我们发现温度、电压和失效率参数都需要被维护。（论文给了 Protégé 对某个温度参数为何需要维护的解释输出，那段解释把整条推理链——从 writeError hasRootFailure HCI、HCI Type AgingFailure、AgingFailure SubClassOf TemporalFailure、TemporalFailure SubClassOf TechnologicalFailure 一直到规则 5 的匹配——逐条列了出来。）

变换操作的规定方式是：实现 OWLAPITransformation trait 的一个子类。为扩大温度区间，他们实现了一个把区间下限调低、上限调高的变换。执行时，先把输入的 MPFO 文档映射成本体表示。

变换前后的本体度量（表 II）：

特征	输入本体	输出本体
公理数	3874	3633
逻辑公理	2889	2773
声明公理	911	860
类数	52	36

特征	输入本体	输出本体
对象属性	39	31
数据属性	30	19
个体	775	774
对象属性断言	779	779
数据属性断言	1216	1216

Even though some axioms have been removed in the transformation, the object and data property assertions are preserved. The loss of some axioms is due to intentionally discarded MP information such as operating states and ports specifications.

尽管变换中移除了一些公理，对象属性断言与数据属性断言都被保留了下来。部分公理的丢失，是因为有意丢弃了某些 MP 信息，比如工况状态和端口规格。

B. 子部件 MP 的推导

这个用例描述如何从一份基准 MP 推导出子部件的 MP。以汽车**电子控制单元（ECU）**的开发为例：Tier-1 组装 ECU 模块，Tier-2 为 ECU 提供集成电路（IC）部件，而该 IC 又由 Tier-3 提供的工艺制造。

In our example, the ECU is mounted in the engine compartment. Using a publicly available typical temperature load distribution one can define a MP for the ECU. This temperature profile of the inner air temperature of the ECU is expressed as a histogram of 12 temperature ranges between -40 ° C and 125 ° C.

在我们的例子里，ECU 安装在发动机舱内。利用公开可得的典型温度载荷分布，可以为 ECU 定义一份 MP。ECU 内部空气温度的这份温度剖面，表示为 -40° C 到 125° C 之间 12 个温度区间的直方图。为了给 Tier-3 提供 MP，Tier-1 必须变换这份温度剖面 MP 数据。

JEDEC 标准 JESD51-2 定义了自然对流条件下测量 IC 热阻的方法。器件结与环境空气之间的热阻由公式 6 确定；写成公式 7 的形式后，给定环境温度 T_A、器件总功耗 P、以及结到环境空气的总热阻 θ_{JA}，就能算出结温 T_J：

$$\theta_{JA} = (T_J - T_A) / P \quad \text{..... (6)}$$

$$T_J = T_A + (P \times \theta_{JA}) \quad \text{..... (7)}$$

$$\theta_{JA} = \theta_{JC} + \theta_{CA} \quad \text{..... (8)}$$

其中 θ_{JC} 是结与器件壳之间的热阻，θ_{CA} 是器件壳与环境空气之间的热阻。这两个公式在工业界也用于计算结温，本例用它们基于 ECU 的 MP 算出 Tier-3 对应的温度剖面。

The actual specification of the transformation operations requires expert knowledge about the specific relationships between the loads of the base component and the sub-component. Parameters which are not identified as required can be stripped off the base MP to protect IP

and reduce required storage space. As MPs are required for each sub-component of the ECU the transformation operations specification can be reused thus saving time and avoiding errors.

变换操作的实际规定，需要关于「基准部件与子部件的载荷之间的具体关系」的专家知识，这些关系体现在变换里，结温计算就是这类关系的一个例子。**未被识别为必需的参数可以从基准 MP 中剥离掉，以保护知识产权并减少所需存储空间。** 由于 ECU 的每个子部件都需要 MP，变换操作的规定可以被复用，从而节省时间、避免错误。

完整流程（假定失效模型与变换本体已含有供应链结构信息和对应的失效模型）：

1. 用户在失效模型与变换本体中指定这次变换所处的供应链层级，比如用 OWL 本体编辑器写一个个体：`example-transform-oem`，类型 `SWRLAPITransformation`，事实 `ofSupplyChainLevel oemLevel`。
2. 系统用失效模型与变换本体、配合任意 OWL 推理机，识别出需要维护的参数。待维护参数会显示为通过 `maintainsParameter` 属性与该变换实例相关联的对象（例子里推出了 `MPParam_failureRate`、`MPParam_v0/v1/v2`、`MPParam_t0/t1/t2` 共七个）。
3. 用户通过实现 `MP0Transformation` trait 或其扩展 trait 来规定变换操作，包括设定变换的数据源与数据目标。公式 7 和 8 就在这里被用来实现对应的变换操作。
4. 系统把 MPFO 格式的 ECU 基准 MP 映射成一个 OWL 本体。
5. 系统最后执行该变换，产出 IC 的 MP 本体。

六、结论与限制

We suggested a transformation system for requirements represented by MPs which is capable of considering the supply chain with respect to high level and technology failure models. The extensible design allows for the integration of multiple transformation approaches which was identified as an important requirement for a MP transformation system.

我们提出了一个针对「以 MP 表示的需求」的变换系统，它能够就高层失效模型和技术失效模型来考量供应链。可扩展的设计允许集成多种变换方法——而这被我们识别为 MP 变换系统的一条重要需求。

A limitation of the SWRL rule-based transformation is that SWRL supports monotonic inference only. Because of that SWRL-based transformations can only add more facts to an ontology but neither modify nor remove existing information. This could be mitigated for example by adding pre- and post-processing steps to SWRL transformations.

基于 SWRL 规则的变换有一个限制：SWRL 只支持单调推理。 因此，基于 SWRL 的变换只能往本体里添加更多事实，既不能修改也不能移除已有信息。这一点可以通过给 SWRL 变换添加前置和后置处理步骤来缓解，在这些额外的处理步骤里处理事实的修改或移除。

Another limiting factor which is related to the current implementation of the component which maps MPs to ontological representations is that the ontology model is constructed in memory.

This is problematic for very large MPs which do not occur so frequently in practice.

另一个限制因素与当前实现有关：把 MP 映射成本体表示的那个组件，[本体模型是在内存里构建的](#)。这对特别大的 MP 会有问题，不过这类 MP 在实践中不那么常出现。

There are also opportunities to improve the approach. General improvements could consist of approaches for reuse of M2M transformations and verification thereof. An improvement with regard to the set of supported transformation approaches could be integration of pattern-based transformation. Further research can also be conducted on direct failure model evaluation using SWRL Built-In extensions.

改进方向也存在。一般性的改进可以包括 M2M 变换的复用方法及其验证方法。就所支持的变换方法集合而言，一项改进可以是集成[基于模式的变换](#)。进一步的研究还可以在「用 SWRL 内置扩展直接做失效模型求值」这个方向上开展。

We considered other technical solutions upfront such as using the Eclipse Modeling Framework (EMF) instead of ontologies and model transformation languages such as the Epsilon Transformation Language (ETL) or the Atlas Transformation Language (ATL) but decided to keep the system design simple in order to reduce complexity.

[我们事先考虑过其它技术方案](#)，比如用 Eclipse Modeling Framework (EMF) 代替本体，以及 Epsilon Transformation Language (ETL)、Atlas Transformation Language (ATL) 这类模型变换语言，**但为了降低复杂度，我们决定保持系统设计简单**。事实证明这是实现 MP 变换系统的一条可行路径，也印证了文献中此前的结果。我们使用了标准化的语言和格式（以及正在标准化过程中的格式），并展示了该方法的实际可用性。

参考链接

- [arXiv:2608.21945](#) — 本文的 arXiv 版本（2026-08-22 提交，但内容是 2019 年 IEEE ISSE 的录用稿）
- [DOI 10.1109/ISSE46696.2019.8984265](#) — 正式发表版本，2019 IEEE International Symposium on Systems Engineering
- [MPFO — Mission Profile Format](#) — autoSWIFT 项目推动的 MP 标准格式
- [RESCAR2.0 项目](#) — MP 格式初版草案的来源
- [autoSWIFT 项目](#) — 推动 MPFO 标准化
- [OWL 2 Web Ontology Language](#) — W3C 标准，本文的模型表达语言
- [SWRL — Semantic Web Rule Language](#) — 本文用于推断待维护参数的规则语言，只支持单调推理
- [Protégé](#) — 本体编辑器，论文用它展示推理解释
- [Play Framework](#) — Semantic MPAD Platform 的实现框架（Scala）